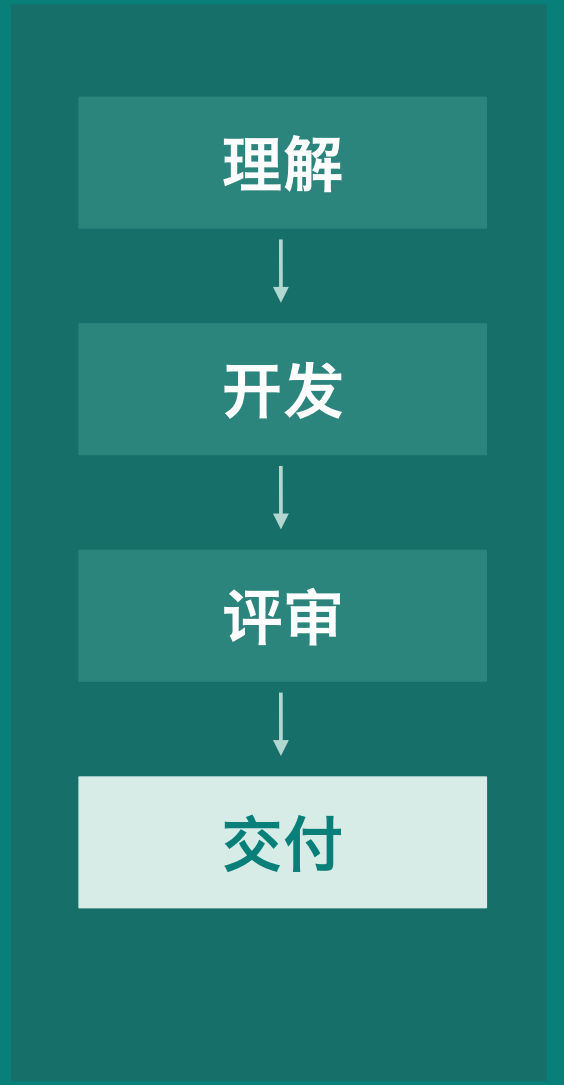
从 AI 写代码
到 AI 协同研发

Zan Workflows 工作流实践

把研发经验固化成流程
让每一步交付都有依据

cfj · 研发团队分享



为什么需要一套工作流

代码生成越来越方便，协作与交付仍需要清楚的约定。

01 / 上下文

每次都从头解释

需求在哪个项目?
现有能力能复用多少?
历史上踩过哪些坑?

02 / 协作

步骤之间容易脱节

方案与实现脱节
多 Agent 修改相互影响
人工改动后评审失效

03 / 交付

完成口径不一致

代码完成 ≠ 验证完成
构建提交 ≠ 部署成功
交接资料与实际变更不一致

目标：减少重复准备，让阶段之间的事实和成果能继续使用。

整个工作流长什么样

需求与 Bug 共享研发主线，准备深度和记录方式按实际工作调整。

01	02	03	04	05	06
读取与定位	**范围与方案**	**实施开发**	**验证与评审**	**测试交付**	**明确上线**
需求／Bug 项目与原分支	清单确认 按需 Task / Plan	固定分支 独立步骤可并行	行为证据 独立最终评审	dev / develop 构建、Wiki、登记	原分支进主线 核验适用发布

同一份范围与上下文 → 同一原分支 → 同一批交付证据

普通开发／修复默认包含提测，先展示完整清单并确认；上线须明确发起。

插件体系如何支撑这条流程

Plugin 组织能力， Skill 描述做法，工具负责实际操作。

ZAN WORKFLOWS PLUGIN

统一安装、更新与能力组织

Skills
阶段规则与调用顺序

Scripts
执行、校验与读回

Hooks
会话衔接与 Git 检查

↓

MCP 连接： TAPD · GitLab · Jenkins · YApi · Graphify Hub

EXTERNAL CONTEXT

外部配套

project-knowledge
项目规则、映射与知识

ai-session-memory
历史决定与失败线索

插件提供工作流与连接能力；知识数据、远端系统和记忆服务各自维护。

日常先记住三个入口

用目标发起工作，由当前阶段按需调用其他能力。

需求开发

`$developing-requirement`

一条 Story 或选定 Task
先核实范围，再开发与交付

Bug 修复

`$fixing-bug`

一条或一组 Bug
复现、修复、评审与溯源记录

直接提测

`$submitting-for-test`

已经有实现成果
核实当前变更后交付

“先完成代码，我检查后再提测”可以明确把本轮终点设在实施与评审。

AI 怎么知道项目背景

不同来源解决不同问题，也有不同的核实方式。

维护中的规则与映射	项目归属、工程约束、服务与 Jenkins 映射	从工作区与权威维护来源核对
项目知识与 Graphify	功能入口、代码关系、影响范围	优先知识入口；快照结合当前源码
会话记忆	历史决定、失败尝试、需要留意的点	小范围检索；候选记忆继续查证
当前源码与系统回执	实际实现、准确版本、写入后的状态	作为当前行为与完成结论的证据

上下文帮助定位和理解，当前事实支撑操作与结论。

用一个案例贯穿全程

订单列表新增状态筛选；切换状态后回到第一页。

教学示例 · 非真实项目与 TAPD 条目

订单列表

全部　**待付款**　已完成

第 1 页 / 筛选结果

把验收写成可观察的行为

01 选中状态后，列表正确过滤

02 切换状态后，页码重置为 1

03 空结果与返回列表行为一致

后续每个阶段都回到同一组行为和验收，避免实现过程中丢失目标。

开发前，先把本轮工作讲清楚

一张清单连接业务范围、实施方式和本轮交付。

项目	分支	交付范围	实施方案	验收标准	需求待确认
准确仓库 与模块	新建／复用 固定开发分支	筛选交互 页码联动 明确排除项	复用列表能力 补充状态参数	筛选、翻页 空结果、返回	仅列业务决定 无则明确无

确定清单 展示完整范围与拟执行动作 → **执行清单** 确认后按同一内容推进

把合并、测试环境、 Wiki 等实际动作一并展示；实质变化再补受影响确认。

小改动和复杂需求怎么处理

按实际需要选择准备深度，保留必要验证和独立评审。

范围集中、行为清楚

简短执行说明

行为与排除项 · 现有证据
实施步骤 · 验证入口与预期

需要正式计划或任务协调

Task + 正式 Plan

真实任务与责任 · 已核实工时
步骤依赖 · Task 绑定 · 验收证据

简单业务决定：对话确认

复杂交互与多组决定：按需评审面板

已有正式 Task ／ Plan 时延续其约束；轻量说明不能绕过尚未满足的条件。

多 Agent 如何协作开发

按依赖和修改边界安排并行，主 Agent 控制集成与共享操作。



共享文件、公共契约、 Git 与交付操作由主 Agent 统一协调。

只有已证实独立的工作才并行；小任务可以直接完成。

怎样判断“做完了”

把完成结论拆成具体事实，避免用一个“通过”覆盖所有阶段。

01 实现完成

当前变更符合已确认范围

准确代码版本与实际 diff

02 行为验证通过

预期和异常路径有证据

定向测试、复现或浏览器结果

03 独立评审通过

审查当前完整变更与影响

与当前版本一致的评审结论

04 交付动作完成

系统接受或状态读回符合预期

MR 、队列、 Wiki 与登记回执

代码变化后，只刷新失效的验证与评审；没有运行或条件缺失就如实记录。

接口没准备好，怎么继续

业务确认、外部条件就绪和执行授权分别判断。

已确认的业务

保留已定行为

筛选项与默认值
切换后的页码行为
空结果和返回逻辑

外部条件与缺口

准确记录依赖

字段含义与接口契约
当前环境与来源
补齐条件和受影响步骤

本轮可执行步骤

按阶段继续推进

先做已就绪的步骤
补齐后沿原任务续接
真实联调单独验收

Mock 默认本地；测试环境需明确范围授权；生产禁止携带 Mock 。

接口未知时使用功能数据场景，不猜 HTTP 契约； Mock 结果不代表真实联调通过。

提测交付包含哪些事情

让代码、交付链接与测试上下文一起到位。

提交／推送

核实当前成果

→

MR 与合并

准确 dev / develop

→

测试构建

核实队列被接受

→

Wiki 与 TAPD

正文、回链与读回

提测阶段的准确结论

“构建任务已提交”

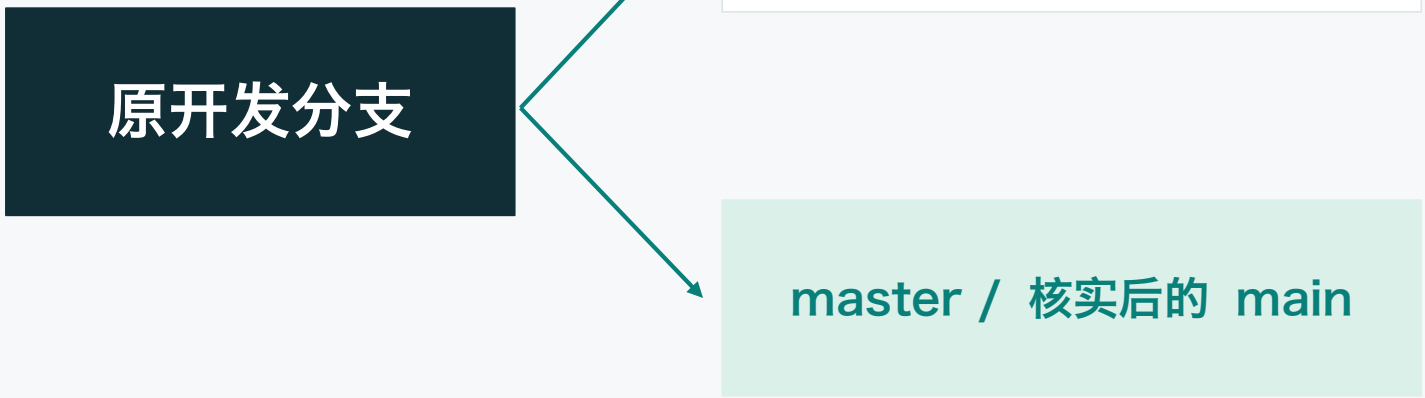
记录准确队列链接后
继续 Wiki 与适用 TAPD 登记。

不等待构建启动或完成。

构建任务被接受，并不能证明测试环境已部署成功。

上线与提测有什么区别

明确发起上线，沿原开发分支进入已核实的默认主线。



测试交付

提交测试构建与测试上下文

正式主线交付

需要发布时核验结果与来源
涉及工具包时检查版本和消费者

主线合并、包发布和生产部署分别核验；主线已合并不能替代发布证据。

人工检查或流程中断后怎么续接

保留原上下文与已完成成果，从首个未完成阶段继续。

人工修改了代码	**核实实际变化**	补受影响验证与独立评审
没有主线合并权限	**保留同一个 MR**	请有权限同事合并，读回后续接
接口或环境后来补齐	**延续原 Task / Plan / 分支**	刷新失效事实与新增范围授权
写入结果未知	**先查系统当前状态**	避免盲目重试和重复创建

续接的目标是复用已验证成果；范围和版本变化时再刷新对应证据。

现场演示：三句话走一个闭环

沿同一个教学案例，观察每一步输出与停在哪里。

01　先实现，我检查

`$developing-requirement`

需求：〈TAPD 链接〉
先完成代码和验证，
我检查后再提测。

观察：范围清单、实际变更、验证与评审

02　检查后直接提测

`$submitting-for-test`

代码检查完成，直接提测。
沿用原分支，先展示具体交付计划。

观察：人工改动复核、MR、队列和 Wiki

03　明确要求上线

`$going-live`

这个变更已经通过测试，请安排上线。
先展示主线交付与适用发布计划。

观察：准确目标、版本核验、未完成项

演示用预先准备的脱敏记录；第三步以展示计划为止，是否执行另看实际授权。

团队如何开始使用

先把连接与上下文准备好，再从一条小任务验证完整链路。

01

准备环境

安装插件与运行依赖
配置个人凭据
审阅 Hook 与核验连接

02

补齐项目上下文

工作区与项目规则
项目知识与准确映射
确认实际工具权限

03

选择一个小任务

目标与验收清楚
走完一次开发与交付
记录断点和改进建议

先验证项目与权限是否匹配；本机配置检查通过，不代表远端权限已经可用。

这套实践值得带走什么

把经验固化成流程
把交付落到证据

可复用

规则、知识与阶段能力

可续接

原任务、原计划、原分支

可核实

验证、评审与系统回执

讨论：选哪一类小任务，作为团队的第一次完整实践？

附录 A | 完整技能速查

按阶段定位能力，日常不需要一次加载全部技能。

发起工作	`developing-requirement` `fixing-bug` `submitting-for-test`
准备与实施	`preparing-work` `writing-plans` `breaking-down-requirement` `implementing-work`
交付与维护	`going-live` `running-release` `merge-request-review` `drafting-wiki` `linking-tapd-wiki`
上下文与辅助	`workspace-project-knowledge` `project-memory-context` `team-identity-map` `login-token-workflow`

准确职责与使用条件，以插件当前 README 和对应 SKILL.md 为准。

附录 B | 安装与首次连接

以下命令来自插件当前 README，并用本机 CLI 帮助核对语法。

```
codex plugin marketplace add cfj1996/skills --sparse .agents/plugins
codex plugin add zan-workflows@cfj-skills
```

运行依赖

Node.js 22 · Volta · uv/uvx
按各 MCP 要求准备运行时

个人配置

~/.config/zan-workflows/
credentials.json

首次核验

核对来源与插件状态
重启、新建聊天、审阅 Hook

project-knowledge 是外部配套仓库；首次初始化与项目规则按工作区维护要求执行。

附录 C ｜常见问题与事实边界

用于现场问答，也作为后续维护这份分享的检查点。

每次都要建 Task / Plan 吗?	按实际需要；已有正式任务与计划继续沿用其约束。
AI 会直接提交和上线吗?	实际动作先进入完整清单确认；上线须明确发起。
构建任务已提交等于部署成功吗?	不等于。提测核实队列接受，正式发布核验结果与来源。
接口没好能用 Mock 吗?	默认仅本地；测试需明确范围授权，生产禁止携带。

事实依据：插件 README；各阶段 SKILL.md； adaptive-workflow、 execution-checklist、 external-data-dependencies、 mock-development 等维护文档。

材料口径： 2026-10-26 的当前仓库说明。案例为教学示例，未使用真实客户或凭据数据。